

ökkoteam



4K+

Охват за 30 дней

Okko

4,6 Оценка работодателя

58,98 Рейтинг

62 398 Подписчики

Подписаться



rav_2549

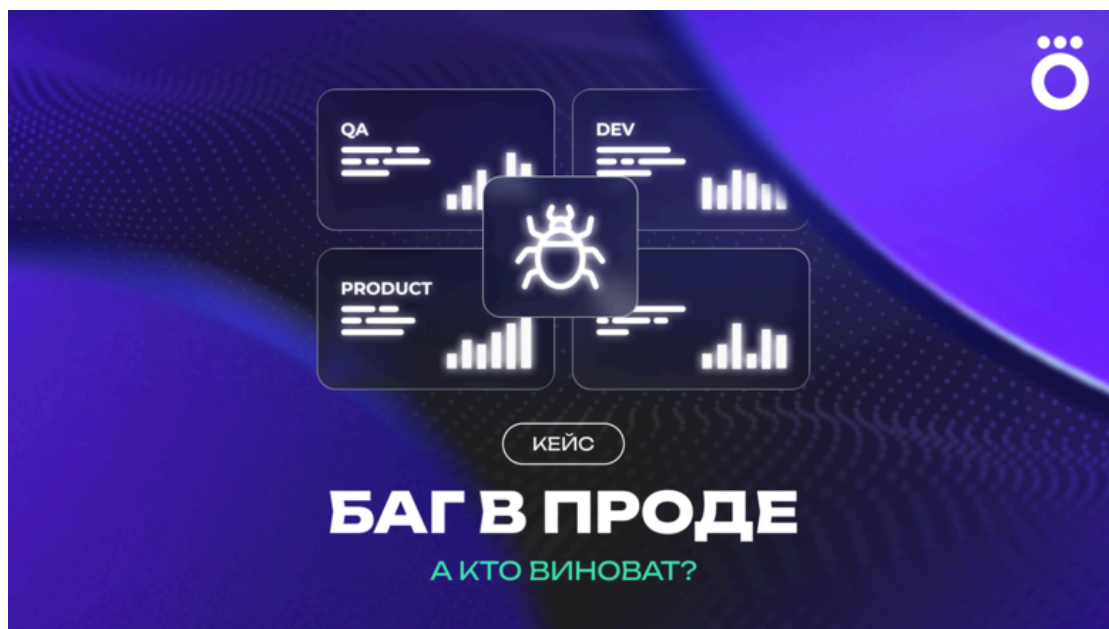
15 часов назад

Баг в проде – а кто виноват? Делим баги на категории, чтобы сделать их заметнее

Простой 11 мин 5.7K

Блог компании Okko, Блог компании Конференции Олега Бунина (Онтико)

Кейс



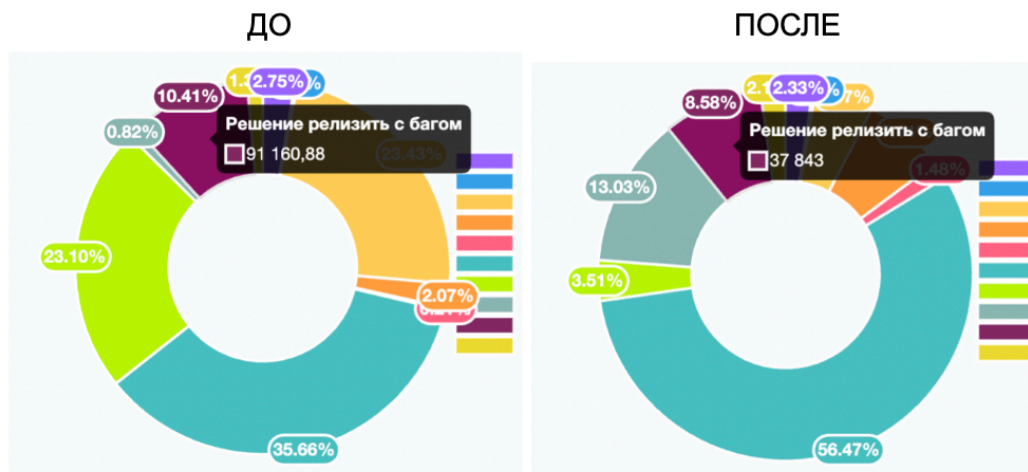
Привет! Меня зовут Андрей Романюк, я руководжу группой качества в онлайн-кинотеатре Okko. В прошлой [статье](#) я рассказывал о том, как мы начали взвешивать баги, сам подход к этому взвешиванию, и о том, что это помогло нам сократить количество заранее известных багов, которые мы выкатываем в прод.

Эта статья о том, как мы решили выделить категории багов, чтобы сделать их заметнее для команд и попробовать на них повлиять. Расскажу, на какие именно категории делили, что это нам дало, и что в итоге получилось.

Предыстория



Когда мы только начали оценивать баги, довольно быстро увидели любопытный эффект. У нас почти перестала появляться одна конкретная группа. Причина оказалась простой: как только мы начали отдельно смотреть на такие случаи и подсвечивать их, на них обратили внимание все участники процесса. Те, из-за чьих решений эти баги раньше доезжали до прода, по сути сами скорректировали поведение: «А зачем мы вообще это выпускаем? Давайте не будем». В результате количество таких багов заметно сократилось.



общий вес багов категории в проде снизился более чем на 50%

Тогда у меня возникла следующая мысль: если сработало для одной группы, значит, возможно, имеет смысл так же посмотреть и на остальные. Может быть, есть и другие категории багов, на которые можно влиять — возможно, не так же, а какими-то своими способами. В итоге мы и начали выделять отдельные группы причин, а дальше уже думать, какие изменения в процессах помогут снизить количество багов в каждой из них.

Выделили мы десять категорий, расскажу про каждую.

Ошибки тестирования

Ошибка тестирования — это, по сути, причина по умолчанию. Когда мы заводим баг на продовое окружение, именно эта категория выставляется автоматически.



(QA смотрящий на баг в проде)

Почему так? Потому что в целом внутри процесса почти всегда есть базовое ожидание: если баг доехал до прода, значит, тестирование его не остановило. Наша задача — проверить релиз, найти проблемы до выкатки и не допустить, чтобы они ушли к пользователю. Если баг все-таки просочился, первая и самая очевидная гипотеза — где-то недосмотрели именно мы.

Поэтому «Ошибка тестирования» и остается дефолтным состоянием. А дальше мы уже отдельно разбираемся, действительно ли причина в тестировании, или баг появился по другой причине. После анализа мы либо оставляем эту категорию, либо меняем ее на какую-то другую.

Решение релизить с багом

Эта категория у нас заметно сократилась после появления скоринга. Механика тут довольно простая. Перед релизом мы отправляем письмо: релиз проверен, вот список известных багов, вот их количество, а вот влияние. Люди, которые принимают решение о выкатке, видят эту картину уже не абстрактно, а вполне конкретно.

И дальше нередко происходит очень полезный для качества диалог: вместо «ну вроде нормально, поехали» появляется реакция в духе «давайте сначала это поправим, а потом уже выпустим релиз». За счет этого таких кейсов стало ощутимо меньше.

Но если решение все-таки принимается в пользу релиза, несмотря на найденные дефекты, мы фиксируем это явно. То есть помечаем баг причиной **«Решение релизить с багом»**: проблему мы нашли заранее, она была известна, но нам все равно сказали выпускать релиз в прод именно в таком виде.

Ошибка эксплуатации/инфраструктуры

Следующая причина — **ошибка эксплуатации или инфраструктуры**. Это не самая частая категория, но почти всегда одна из самых болезненных.

Обычно такие баги появляются в ситуациях, когда кто-то со стороны эксплуатации меняет настройки, выкатывает какие-то инфраструктурные изменения или затрагивает конфигурацию фронта, а в результате что-то ломается. Причем ломается, как правило, не локально, а сразу заметно и массово.

У таких инцидентов есть характерная особенность: они встречаются редко, но если уж случаются, то почти всегда вносят очень весомый вклад в общую картину продовых проблем. Потому что это не история про «где-то поехала одна кнопка» — чаще речь идет о поломках, которые затрагивают большую часть функциональности или вообще весь сервис целиком.

Не описано влияние на элементы сервиса

Категорию **«Не описано влияние на элементы сервиса»** мы выделили отдельно после договоренности с разработчиками. Идея была простой: когда задача передается в тестирование, разработчик оставляет комментарий с пояснением, какие компоненты или части системы он затронул и на что QA стоит посмотреть особенно внимательно.

Это не попытка переложить на разработку ответственность за тест-дизайн и не замена документации. Скорее, способ заранее подсветить зоны риска. Потому что на практике задача в треке может быть про одно, а реальные изменения в коде — сильно шире и неочевиднее.

Для QA это критично: если не знаешь, что именно было затронуто под капотом, не всегда можно догадаться, где искать побочные эффекты. В итоге задача вроде бы протестирована, но баг все равно уезжает в прод — просто потому, что проявляется в неожиданном месте. Условно, дорабатывали авторизацию, а сломался просмотр видео. На первый взгляд связи между этими вещами нет, но внутри системы она может быть.

Поэтому, если влияние изменений на элементы сервиса не описано совсем или описано неполно, это становится отдельной и вполне реальной причиной продовых багов.

В требованиях отсутствует или неактуален дизайн

Эту категорию мы ставим в тех случаях, когда проблема связана именно с макетами. Например, если в дизайне есть неточности, ошибки или просто устаревшие решения, а реализованное поведение при этом в целом соответствует тому, что нарисовано.

Сюда же относятся ситуации, когда баг есть, но конкретный пользовательский кейс вообще не был проработан на уровне макетов. То есть для него просто не существовало корректного дизайн-описания: сценарий не учли, экран не продумали, состояние не отрисовали. В таком случае мы тоже относим проблему к этой категории.

В требованиях отсутствует/не актуальная/не полная документация

Причина «**В требованиях отсутствует, неактуальная или неполная документация**» — по сути, очень похожая история, только уже не про визуальную часть, а про требования и текстовое описание логики. Если нужный сценарий не был задокументирован, был описан не полностью или документация на момент реализации уже устарела, это тоже становится отдельной причиной появления бага в проде.

В отличие от проблем с дизайном, здесь мы говорим уже не про макеты, а про **логику системы**. Категорию используем тогда, когда баг возникает в кейсе, который изначально никто не проработал и не описал.

Иными словами, это ситуация, в которой сценарий не был зафиксирован в требованиях: команда не предусмотрела его заранее, не описала ожидаемое поведение и не дала тестированию опоры для проверки. Поэтому такие баги правильнее ловить не на этапе релиза, а гораздо раньше — во время проработки требований и подготовки документации.

Тестирования перед включением не было

Мы используем ее в тех случаях, когда фича какое-то время была выключена, а потом ее просто вернули в прод без дополнительной проверки. Типичный сценарий выглядит так: когда-то функциональность работала нормально, потом ее отключили — например, потому что она стала неактуальной, требовала доработки или просто временно была не нужна. После этого она могла полгода или даже год находиться в выключенном состоянии.

За это время в системе многое меняется: дорабатываются соседние части продукта, меняется логика, обновляются зависимости, может меняться поведение связанных компонентов. И хотя в саму выключенную фичу никто напрямую не вносил изменений, это вовсе не значит, что после повторного включения она по-прежнему будет работать корректно.

При этом мы ее все это время не проверяем — и это нормально, потому что она выключена и не влияет на пользователя. Но именно поэтому перед повторным включением такую функциональность нужно обязательно отдельно тестировать: убедиться, что она все еще работает так, как ожидается.

Если этого не сделали, фичу просто включили, а потом в ней нашли баги, мы фиксируем причину **«Тестирования перед включением не было»**. По сути, это сигнал: перед тем как возвращать что-то в прод, нужно сначала проверить, не сломалось ли оно за то время, пока было выключено.

Решение релизить без тестирования

Еще одна, уже более редкая категория — **«Решение релизить без тестирования»**.

Такие ситуации случаются нечасто, но все же бывают. Например, когда у нас нет свободного ресурса на тестирование, не хватает времени или задачу нужно выпустить очень быстро. В этот момент кто-то со стороны продукта или разработки может сказать: «Здесь ничего опасного не менялось, мы сами посмотрели, дополнительное тестирование не нужно».

Иногда это действительно проходит без последствий. Но если после такого решения в проде обнаруживается баг, мы отдельно фиксируем именно эту причину. То есть проблема здесь не в том, что QA что-то пропустил, а в том, что релиз сознательно отправили без полноценной проверки.

В таких случаях мы и ставим категорию **«Решение релизить без тестирования»**. Это прозрачная фиксация факта: команде тестирования не дали возможности проверить изменение, но релиз все равно состоялся — и в итоге это привело к багу в проде.

Внешние изменения

Отдельно мы выделили категорию **«Внешние изменения»**. Сюда относится все, что приходит к нам извне: обновления браузеров, новые версии Windows или других операционных систем, выход новых iPhone, Android-устройств с нестандартными экранами — например, раскладушек, — и любые другие изменения платформ, которые мы не контролируем.

Логика здесь простая: мы не всегда можем заранее предусмотреть, что именно изменится в новой версии системы или как наше приложение поведет себя в новом окружении. Особенно если речь идет о совсем свежих релизах, бета-версиях или новых форм-факторах устройств.

В таких случаях проблема возникает не потому, что мы плохо протестировали уже известный сценарий, а потому, что внешняя среда изменилась сама по себе. Вышло что-то новое, и именно на нем приложение начинает работать некорректно — хотя на всех предыдущих версиях и устройствах все могло быть нормально.

Такие кейсы у нас тоже встречались. Например, при переходе на новые версии ОС нередко всплывают неожиданные эффекты, которые приходится отдельно отлавливать и разбирать еще на этапе беты. И если баг проявился именно из-за такого внешнего изменения, мы фиксируем его в этой категории.

Ошибка редактора

Сюда мы относим баги, которые на самом деле связаны не с кодом и не с логикой приложения, а с некорректно заведенным контентом. Речь про случаи, когда редакторы добавляют в админке фильмы, сериалы, карточки контента, изображения, описания и другие связанные данные, и где-то на этом этапе допускается ошибка.

Например, может оказаться, что загрузили не ту картинку, указали неверный текст, перепутали серии или допустили другую неточность в заполнении. В результате в приложении все отображается некорректно, хотя технически сама функциональность может работать правильно.

То есть причина здесь не в том, что что-то сломалось в продукте, а в том, что контент был неправильно заведен со стороны редакторов. Если баг возникает именно по этой причине, мы относим его к категории **«Ошибка редактора»**.

Что дальше? Работаем с категориями

После того как мы выделили группы, в Jira появилось отдельное поле — **Reason**. Мы начали заполнять его для каждого бага, который заводится на продовое окружение.

По умолчанию в этом поле стоит значение **«Ошибка тестирования»**. И логика здесь понятная: почти у любого участника процесса доставки кода в прод есть базовое ощущение, что если баг проявился уже у пользователя, значит, на этапе тестирования его просто пропустили. В каком-то смысле это действительно дефолтная гипотеза.

Но дальше QA уже может поменять причину на другую — только не «по ощущению», а после обязательного разбора. Если тестировщик считает, что баг появился не из-за ошибки тестирования, он должен зафиксировать это в комментарии к задаче и объяснить, почему относит проблему к другой категории.

Дальше мы какое-то время просто копили данные: продолжали заводить баги, раскладывали их по группам и параллельно разбирали продовые баги за несколько предыдущих месяцев, чтобы быстрее набрать статистику.

Когда у нас накопились данные примерно за полгода, мы снова собрались и задали себе следующий вопрос: а можем ли мы повлиять не только на одну уже заметную группу, но и на другие? Можем ли сделать так, чтобы они тоже начали снижаться?

Оказалось, что да!

Примеры решения проблем

Когда мы собрали статистику, стало понятно: на часть групп мы действительно можем влиять вполне предметно.

1. Ошибка тестирования

Это как раз та категория, на которую мы, как QA, можем влиять напрямую. Здесь у нас в руках и процессы, и техники, и правила, и подходы к проверкам.

Если же говорить про категорию **«Ошибка тестирования»**, то тут пространство для изменений вообще огромное. На нее можно влиять настолько широко, насколько хватает фантазии и ресурса. Например:

- увеличивать покрытие автотестами;
- делать предрелизные проверки глубже;
- уменьшать размер релиза, потому что чем меньше изменений уезжает за раз, тем ниже шанс что-то сломать;
- переходить от чек-листов к полноценным тест-кейсам, чтобы проверка была детальнее и меньше зависела от контекста в голове конкретного человека.

Из интересных практик, которые сработали у нас, была внутренняя активность под названием **«пятнадцатиминутка»**. Суть в том, что вся команда одновременно брала устройства, открывала приложение в продовом окружении и в течение короткого времени смотрела какой-то конкретный функционал. Что именно проверять, мы обычно решали прямо перед началом.

Эта практика стабильно приносила обнаруженные баги. Во многом потому, что люди переключали контекст, смотрели на продукт свежим взглядом и замечали вещи, которые легко пропустить в рамках привычной работы. Причем находились не только мелкие шероховатости, но иногда и вполне серьезные проблемы.

В какой-то момент мы подумали: если эта активность так хорошо работает, почему мы делаем ее уже на проде, а не раньше? Почему бы не перенести тот же самый подход на пре-прод?

В итоге так и сделали. И перенос «15-минутки» на другое окружение действительно дал эффект: мы стали чаще ловить баги еще до того, как они доезжали до прода.

2. Не описано влияние на элементы сервиса

Здесь у нас получилось довольно быстро договориться с разработкой.

Мы просто пришли к ребятам и честно объяснили логику: если разработчик в рамках задачи затронул что-то неочевидное, а тестирование об этом никак не узнало, то потом такой баг легко может выглядеть как «ошибка тестирования», хотя по сути проблема не в этом. Поэтому мы предложили простой подход: при передаче задачи в тестирование коротко писать, что именно было затронуто и куда нам стоит посмотреть внимательнее.

Что для меня было важно — никакого сопротивления здесь вообще не возникло. Не было реакции в духе «зачем нам это надо» или «это лишняя бюрократия». Наоборот, ребята очень хорошо это приняли и довольно быстро встроили в работу.

В итоге договорились о следующем: при передаче задачи в тестирование разработчик оставляет комментарий, где описывает, что именно он затронул в рамках реализации и какие части системы, по его мнению, могли зацепиться дополнительно. Это не замена документации и не попытка переложить на разработку тест-дизайн, а именно подсветка зон риска.

3. Ошибка редактора

Здесь мы пошли в сторону автоматизации и сделали валидатор данных. Он проверяет, что все обязательные поля заполнены и что данные в них корректны. За счет этого часть проблем начала отсеиваться еще до того, как некорректный контент попадал в приложение.

4. Тестирования перед включением не было

Здесь снижение тоже произошло во многом за счет прозрачности, почти как в категории **«Решение релизить с багом»**. Решение включить фичу в прод без дополнительного тестирования обычно принимает продакт. И когда после такого включения появляется баг, причина в задаче видна явно.

По сути, механизм сработал очень похоже: как только источник проблемы зафиксирован, он становится заметен, сами такие решения начали приниматься гораздо реже. Сейчас включение чего-то без предварительной проверки — это уже скорее исключение.

Результаты

В итоге вся эта работа дала нам не просто красивую классификацию багов. Главное было в другом: мы разделили родовые проблемы на категории и начали точно влиять на каждую из них своими способами.

И, наверное, самый важный результат — именно тот, ради которого все и затевалось, — это заметное снижение общего количества багов в проде. Причем снизилось число и тех проблем, которые мы раньше просто пропускали и которые «внезапно» всплывали уже у пользователя, и тех, которые были известны заранее, но по разным причинам все равно доезжали до прода.

За счет такой детализации мы стали лучше понимать, откуда именно берутся баги, а значит — быстрее и точнее влиять на причины, а не просто бороться с последствиями.

Если смотреть шире, то главный эффект для меня здесь в том, что сервис в целом стал качественнее и приятнее для пользователя. Стало меньше и крупных проблем, и мелких шероховатостей, которые по отдельности могут казаться нестрашными, но в сумме очень сильно влияют на восприятие продукта.

Потому что пользователь обычно не делит проблемы на критичные и некритичные так, как это делаем мы внутри команды. Для него орфографическая ошибка, странный отступ, наезжающие друг на друга элементы или другая мелкая визуальная неровность — это все равно сигнал, что с сервисом что-то не так. Даже если формально все работает.

Таких вещей у нас стало заметно меньше. А значит, у пользователя стало меньше поводов воспринимать сервис как сырой, кривой или недоделанный.

Можно ли отсюда провести прямую линию к росту базы, подписок и вовлеченности? Наверное, нет — по крайней мере, я бы не стал утверждать это слишком категорично. На такие метрики влияет слишком много факторов.

Но я точно могу сказать, что качество продукта — это одна из составляющих такого роста. Когда сервис работает стабильнее, выглядит аккуратнее и реже раздражает пользователя мелкими и крупными багами и сбоями, это в любом случае идет продукту в плюс.

Теги: баги, bugs, тестирование, влияние, qa, production, jira, процессы тестирования, zbp

Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Электронпочта

Подписаться

Оставляя почту, я принимаю [Политику конфиденциальности](#) и даю согласие на получение рассылок



4K+

Охват за 30 дней

Okko

[Сайт](#) [Сайт](#) [Сайт](#)



4K+

3

Охват за 30 дней

Карма

Андрей Романюк [@rav_2549](#)

Пользователь

Подписаться



Комментировать

Публикации

ЛУЧШИЕ ЗА СУТКИ

ПОХОЖИЕ



the_bat

23 часа назад

Ремонт блока питания с Power Delivery. 470 граммов электроники

Простой

4 мин

16K

+96

30

27



interpres

18 часов назад

Электроинструмент становится хуже, и это делается намеренно

Простой

11 мин

27K

Обзор

Перевод

+71

45

83



prohronus

19 часов назад

Как ИИ-агенты стали новым оружием скамеров на Хабр Карьере

🕒 5 мин 👁 9.3K

Кейс

📦 +51

🔖 9

💬 9



alizar

22 часа назад

Готовимся к отключению. Эффективные форматы для упаковки и раздачи HTML-страниц

🏷 Простой 🕒 7 мин 👁 12K

Обзор

📦 +37

🔖 59

💬 18



AlexSerbul

23 часа назад

Программирование с AI-ассистентом — похороните меня под плинтусом

🏷 Средний 🕒 14 мин 👁 9.9K

Мнение

📦 +28

🔖 33

💬 11



TrexSelectel

19 часов назад

Linux 7.0 и что изменилось в ядре после очередного цикла разработки

🕒 5 мин 👁 8.8K

Обзор

📦 +27

🔖 10

💬 0



mariklolik

18 часов назад

Как переложить нагрузку по code review с разработчиков на LLM

🏷 Средний 🕒 7 мин 👁 7.5K

Кейс

📦 +23

🔖 23

💬 6



nick_dyuba

22 часа назад

Легенды 90-х — кто придумал и производил жвачки Turbo, Love is... и TіріTіr

🕒 5 мин 👁 7.1K

Recovery Mode

📦 +22

🔖 8

💬 5



Mimizavr

23 часа назад

«Великое очищение» в работе с контентом: что осталось от роли редактора



11 мин



7K



+17



9



7



infgotoinf

10 часов назад

Perl — зря забытый язык программирования?



11 мин



8.5K

Из песочницы



+16



13



9

Показать еще

ИНФОРМАЦИЯ

Сайт

okko.tv

Дата регистрации

12 февраля 2019

Дата основания

18 июля 2011

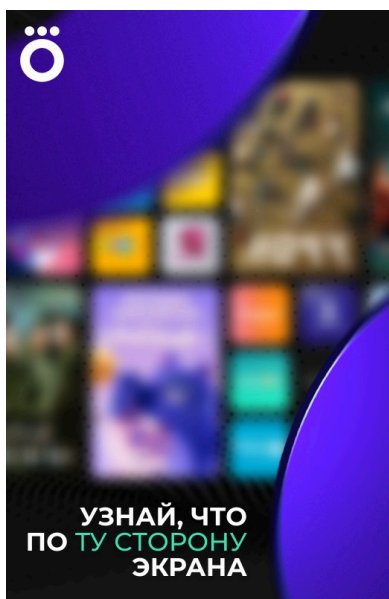
Численность

1 001–5 000 человек

Местоположение

Россия

ВИДЖЕТ



15 часов назад

Баг в проде – а кто виноват? Делим баги на категории, чтобы сделать их заметнее

5.7K 0

21 янв в 14:01

Как бренд команды даёт проекты, людей и влияние

5.9K 0

24 дек 2025 в 16:47

Мой внутренний самозванец: как я учусь жить с тем, кто постоянно говорит, что Я «фейк»

6.9K 3

31 окт 2025 в 17:03

Кастомная сериализация с помощью kotlinx.serialization

6.3K 2

19 окт 2025 в 17:30

Арріт-тесты на ARM и управление девайсами — легко :-)

6.2K 1

Ваш аккаунт

Войти
Регистрация

Разделы

Статьи
Новости
Хабы
Компании
Авторы
Песочница

Информация

Устройство сайта
Для авторов
Для компаний
Документы
Соглашение
Конфиденциальность

Услуги

Корпоративный блог
Медийная реклама
Нативные проекты
Образовательные программы
Стартапам



Настройка языка

Техническая поддержка